TubeBench: A Benchmark for Long-Form Media Interaction in Browser-Use Agents

Venkata Hemanth Athota

June 2026

Abstract

Browser-use agents and AI co-work systems are increasingly asked to operate inside long-form media workflows: finding moments in recordings, controlling players, using transcripts and captions, preserving browser state, and justifying answers from observed evidence. A simple request such as "pause the active player without disturbing anything else" already requires the agent to identify the right tab, mutate only the permitted media state, verify the result, and leave an auditable trace. TubeBench introduces a harness-agnostic benchmark protocol for this class of browser-based media interaction. The current paper reports benchmark design evidence, deterministic diagnostic reference conditions, and static protocol-validation campaigns. Its evidence supports methodology and readiness claims, not broad performance claims for any single model or harness. The positive result is that trace-level scoring preserves hidden failure modes that final-answer scoring would collapse: partial progress, evidence grounding, media navigation, transcript use, tool recovery, verification quality, side effects, final-answer correctness, and trajectory validity.

1 Introduction

Consider a user with several media tabs open: one lecture is paused, another player is active, and a third recording must not be touched. The user asks an agent to pause only the active player and preserve every other player state. A final message saying "done" is not enough. The agent may have paused the wrong player, lost the task tab, skipped verification, or performed the right action without producing a trace that another evaluator can audit.

The static TCE-002 validation task compresses this failure story into a small protocol instance: pause the one playing player while preserving other players. The same structure appears in longer media work, where a user asks an agent to find a claim in a lecture, answer from transcript evidence, compare two recordings, seek to a timestamp, or restore playback state after inspection. These tasks are not reducible to video question answering. They require browser control, temporal evidence, media mutation, protected state, and verification.

TubeBench addresses this setting as an evaluation problem for browser-use agents, AI co-work systems, agentic harnesses, interactive browser agents, and model-tool environments. The central claim is methodological: in long-form browser-media work, the path matters. A final answer alone cannot show whether the agent used the right evidence, changed the wrong player, recovered from a coordinate error, disturbed unrelated state, or lost the task surface before interaction. TubeBench is designed to preserve those hidden failure modes as scoreable evidence.

This paper makes four contributions:

1. a benchmark framing for long-form browser-media workflows that sit between static document QA, simple web QA, coding-only benchmarks, generic GUI tasks, and short-horizon browser interactions;
2. a reusable task and trace protocol covering task objective, media context, browser state, allowed tools, expected evidence, scoring rubric, trajectory capture, and failure labels;
3. trace-level scoring dimensions that distinguish outcome quality from evidence quality, browser/media state control, recovery, verification, side effects, and trace validity; and
4. bounded diagnostic and protocol-validation evidence showing that the evaluation substrate preserves measurement failures instead of compressing them into a single success rate.

2 Related Work and Evaluation Gap

TubeBench builds on web and GUI-agent evaluation while targeting a specific interaction regime. WebArena evaluates long-horizon web tasks in realistic sites [6]; VisualWebArena emphasizes visually grounded web work [3]; Mind2Web studies generalist web agents on real websites [1]; and OSWorld evaluates agents in broader computer environments [4]. SWE-bench motivates executable, evidence-backed evaluation in repository tasks [2]. Recent work on living-screen interfaces also highlights the volatility of media-centered environments [5].

These benchmarks leave an important gap. Static document QA can test reading but not player-state manipulation. Web QA can test navigation but often omits temporal observation and media controls. Coding benchmarks test repository-grounded edits but not browser-mediated evidence gathering. Generic GUI tasks test interface control but may not require transcript use, caption availability checks, timestamp localization, state restoration, or media-specific side-effect accounting. TubeBench focuses on the intersection: agents must navigate long-form content over time, interact with media controls, ground claims in observable evidence, and leave a replayable trajectory for audit.

3 TubeBench Task Protocol

3.1 Task Families

The deterministic fixture suite currently defines twelve task categories that exercise the intended browser-media surface without depending on unstable external pages. The categories cover identifying the active media instance, pausing one or all active players, restoring original playback state, seeking to a timestamp, changing playback speed, muting and verifying, locating a segment from chapters and transcript, answering a timestamp-localized transcript question, finding a visual-only event, comparing two long-form sources, and detecting a restored side effect. These categories are design evidence for benchmark coverage.

The task families are deliberately broader than one website. A TubeBench task can be implemented in a benchmark-owned fixture, a local media-player surface, or an approved public media setting when volatility and privacy constraints are handled separately. The common structure is the interaction contract: the agent must work through browser-visible state and produce a scoreable trajectory.

3.2 Protocol Fields

Each task specification includes:

- objective: the user-facing goal or final answer target;
- media context: videos, transcripts, chapters, visual events, metadata, and relevant time spans;
- browser state: tabs, player states, protected state, and initial conditions;
- allowed tools and actions: observation channels, permitted mutations, and forbidden mutations;
- expected evidence: transcript, caption, metadata, screenshot, player-state, DOM, or visual evidence channels;
- scoring rubric: exact predicates, ordinal or partial criteria, verification requirements, and side-effect checks;
- trajectory capture: observations, actions, browser/tool calls, state snapshots, errors, recovery attempts, and final claims; and
- failure labels: primary and contributing categories for task, grounding, verification, trace, and runtime failures.

TCE-002 is one compact instance of this schema. Its value is not task breadth; its value is that a small media-control request exercises browser-visible state, permitted mutation, verification, trace handoff, retained-slot accounting, and runtime failure labeling.

3.3 Evaluation Pipeline

Figure 1 summarizes the evaluation loop. The pipeline starts with a task specification, instantiates a browser/media environment, executes the agent or harness, captures the trace, scores the trajectory, and feeds error analysis back

into task and rubric design. Raw trajectories remain outside paper claims until reviewed aggregate statistics are available.

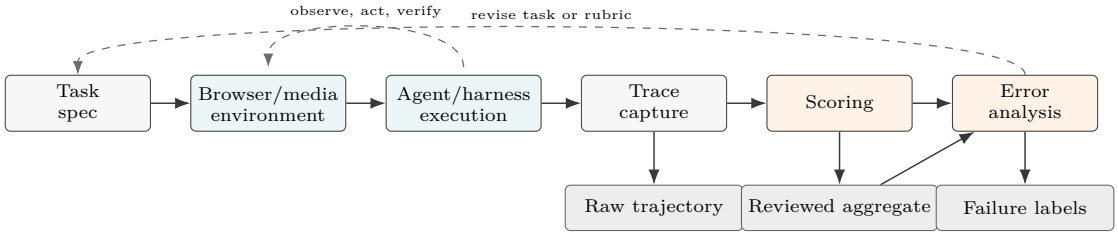

Figure 1: TubeBench evaluation pipeline. Task specifications produce browser/media executions; traces are scored into aggregate metrics and failure labels, then used to refine task and rubric design.

4 Metrics and Trace-Level Scoring

TubeBench avoids reducing browser-media work to a single final-answer score. A run can satisfy the final state while being poorly grounded, can make partial progress before a runtime blocker, or can recover from a mistaken click while still leaving a side-effect incident. Table 1 groups the reporting dimensions by the reader question they answer.

Table 1: Trace-level scoring dimensions grouped for reader interpretation.

Group	Dimension	Reader question
Outcome	Task completion; partial completion; final-answer correctness	Did the requested state or answer actually get completed, and what useful progress survived failure?
Evidence	Evidence grounding; transcript utilization; verification quality	Did the trace show the media, transcript, metadata, visual, or state evidence needed to justify the claim?
Browser and media state	Media navigation; media control; side-effect preservation	Did the agent reach the right surface, operate the right player, and avoid disturbing protected state?
Trace and audit	Tool recovery; trajectory validity; failure label	Can the run be ingested, replayed, scored, and assigned to the right model, task, harness, or runtime failure mode?

For deterministic state predicates, the evaluator records exact success:

$$S_{\text{exact}} = \mathbf{1}[\text{all required final predicates hold}].$$

The stricter disturbance-free score requires success, required evidence, and no forbidden side-effect incident:

$$S_{\text{dfs}} = \mathbf{1}\left[S_{\text{exact}} = 1 \wedge E_{\text{required}} = 1 \wedge I_{\text{side effects}} = 0\right].$$

Exact scoring is useful when the task state is fully specified. It is not sufficient for browser-media evaluation because a scoreable trajectory may include weak grounding, missing verification, partial recovery, trace-transfer failure, or a runtime blocker. Those events must remain visible in the report.

5 Experimental Evidence

5.1 Diagnostic Reference Conditions

The frozen aggregate contains four deterministic diagnostic conditions with 72 attempts each. These rows validate the scorer, aggregation code, and paper pipeline; they are diagnostic controls, not empirical performance measurements for deployed browser agents. The no-op condition executes no task mutation. The three curated references are oracle-assisted reference trajectories generated from the deterministic task catalog and scoring expectations.

Table 2: Diagnostic-only reference aggregate. These rows are pipeline controls for scorer and reporting behavior.

Condition	Attempts	Exact	DFS	Interpretation
Diagnostic-only no-op	72	4.2%	4.2%	Negative control. Its 3/72 successes come from TC-004, where doing nothing is correct state-preserving behavior.
Planner-executor reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.
Skill-augmented reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.
Hybrid reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.

Table 2 is a diagnostic-only table. Its lesson is that the publication pipeline preserves denominators and separates exact task success from disturbance-free success. This matters because a harness may complete a visible task while disturbing protected state, using insufficient evidence, or producing an invalid trace.

5.2 Static Protocol Validation

If Table 2 shows that the scorer and aggregate pipeline behave as expected, Table 3 is the empirical narrative center of the current paper. Two retained TCE-002 static protocol-validation campaigns are recorded as aggregate evidence. The earlier campaign retained five slots: four produced valid task-started traces and one was runtime-blocked before task interaction. The readiness-gated campaign also retained five slots: all five readiness gates passed, but all five slots were blocked before task interaction when the browser controller lost the task tab.

Table 3: Retained static protocol evidence. Readiness passed in the second campaign, but task interaction did not start.

Campaign	Slots	Ready	Started	Valid	Blocked	Reader-facing interpretation
Earlier retained run	5	N/A	4	4	1	The task could start and traces could be scored, but handoff stability was not strict.
Readiness-gated run	5	5	0	0	5	Readiness passed; the failure moved to controller/tab lifecycle before task behavior.

For a retained campaign of N slots, TubeBench reports:

$$r_{\text{ready}} = \frac{n_{\text{ready}}}{N}, \qquad r_{\text{valid}} = \frac{n_{\text{valid traces}}}{N}, \qquad r_{\text{blocked}} = \frac{n_{\text{runtime blocked}}}{N}.$$

For the readiness-gated campaign, $r_{\text{ready}} = 1.0$, $r_{\text{valid}} = 0.0$, and $r_{\text{blocked}} = 1.0$. This is useful because it localizes failure before task interaction. Replacing blocked slots would erase the reliability signal that a browser-media benchmark must preserve.

6 Results and Failure Analysis

The current evidence supports three bounded findings.

Trace-level scoring preserves hidden failures. Exact completion and disturbance-free completion must be reported separately. A run that reaches the requested final state can still be weakly grounded, unverifiable, or unsafe with respect to protected player state.

Diagnostic controls validate the evaluation substrate. The frozen reference aggregate keeps the denominator visible, distinguishes no-op behavior from curated reference behavior, and catches scoring or reporting regressions before agent comparisons are introduced.

Retained blockers identify the failure layer. The static campaigns identify trace and runtime states that are neither task failures nor successes. The readiness-gated run shows a controller/runtime failure before task behavior, which is a different finding from a media-reasoning failure.

Table 4 gives the failure labels used to preserve these distinctions.

Table 4: Failure labels for trace-level browser-media evaluation.

Failure label	Observable signal
Task misunderstanding	Wrong media item, time span, player, or state predicate.
Navigation failure	The agent cannot reach or retain the relevant browser/media context.
Media-control failure	The wrong control is invoked or the wrong player state changes.
Evidence-grounding failure	The final claim is unsupported, partially supported, or contradicted by observed evidence.
Transcript or metadata misuse	Transcript, captions, chapters, or metadata are treated as sufficient when required evidence is absent or conflicting.
Weak verification	The agent declares completion without checking the required state or evidence.
Side-effect disturbance	Unrelated tabs, players, account-visible state, or protected browser state are changed.
Brittle recovery	A mistaken action is partly repaired but leaves state, evidence, or reasoning inconsistent.
Trace-transfer failure	Browser-visible work occurs but the trace cannot be captured, ingested, or scored.
Runtime/controller failure	Browser, tab, controller, or harness lifecycle failure prevents or interrupts task interaction.
Overconfident finalization	The final answer masks partial completion, missing evidence, or infrastructure blockers.

7 Discussion

TubeBench is most useful when an evaluator needs to decide where a failure belongs: the task, the evidence source, the browser state, the agent policy, the controller, or the trace handoff. Long-form browser-media work creates failure modes that are easy to hide behind final-answer scoring: selecting the wrong player, treating a transcript affordance as transcript evidence, confusing metadata with observed media, disturbing another media instance, or reporting success after brittle recovery. Trace-level evaluation turns those events into auditable records.

This design also changes how results should be read. A single top-line score is too small for the domain. Exact predicates remain important for deterministic state tasks, but they have to be paired with grounding, verification, side-effect, recovery, and trace-validity evidence. Ordinal scoring can capture partial progress, but it needs reviewed rubrics to avoid vague success inflation. For live media pages, transcripts, captions, ads, consent flows, recommendations, layout experiments, and live edges can change at run time, so benchmark-owned fixtures and aggregate-only promotion boundaries remain part of the methodology.

8 Limitations

- Current evidence is diagnostic/reference evidence and static protocol validation; it should not be read as a broad model, harness, or leaderboard ranking.
- TCE-002 is intentionally compact. It validates the protocol shape for a state-preserving media-control task, not the full difficulty range of transcript, visual-event, comparison, or long-horizon navigation tasks.
- The curated reference conditions are oracle-assisted diagnostic anchors. They validate scoring and reporting paths but do not represent deployed agent reasoning.
- Live media remains volatile. Public pages can change transcripts, captions, ads, consent flows, layout, recommendations, and playback state during a run.
- Exact predicates can be too narrow if a rubric omits valid alternatives, and ordinal rubrics can be too generous without reviewer discipline.

9 Artifact Availability

The public benchmark repository is available at `https://github.com/heyyymonth/codextubebench`. The published paper PDF is served from `https://heyyymonth.github.io/codextubebench/paper/tubebench.pdf`. The paper uses aggregate, redacted evidence only; raw traces, browser profiles, account context, and authenticated page captures are excluded from the publication artifact boundary.

10 Conclusion

TubeBench frames long-form browser-media interaction as a trace-level benchmark for browser-use agents and AI co-work systems. Its task protocol captures the objective, media context, browser state, allowed tools, evidence channels, scoring rubric, trajectory, and failure labels needed to evaluate agentic media workflows. The current diagnostic and static protocol-validation evidence shows that completion, grounding, media control, recovery, verification, side effects, and trace validity must be reported separately. The methodological contribution is an evaluation substrate that keeps the evidence needed to measure interactive browser agents rigorously.

References

[1] Xiang Deng et al. Mind2web: Towards a generalist agent for the web, 2023. URL https://arxiv.org/abs/2306.06070.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL https://arxiv.org/abs/2310.06770.

[3] Jing Yu Koh et al. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024. URL https://arxiv.org/abs/2401.13649.

[4] Tianbao Xie et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972.

[5] Jiashu Yao, Heyan Huang, Daiqing Wu, Wangke Chen, Huaxi Ai, Haoyu Wen, Zeming Liu, and Yuhang Guo. Benchmarking living-screen-native gui agents on short-video platforms, 2026. URL https://arxiv.org/abs/2606.04701.

[6] Shuyan Zhou et al. Webarena: A realistic web environment for building autonomous agents, 2023. URL https://arxiv.org/abs/2307.13854.